Intelligent OCR System Technical Report

1. OCR Workflow

The application accepts PDF, PNG, JPG, and JPEG documents through the Streamlit frontend and FastAPI backend. PDF files are rendered page by page with PyMuPDF. Each page or image is preprocessed with OpenCV before OCR.

The preprocessing pipeline supports grayscale conversion, optional Gaussian blur noise reduction, deskewing, contrast enhancement with CLAHE, and Otsu thresholding. The processed image paths are returned to the frontend so the reviewer can inspect before-and-after results.

OCR is performed with EasyOCR. The default language is English, and additional EasyOCR languages can be enabled through the comma-separated `OCR_LANGUAGES` environment variable when multilingual OCR is required.

The OCR service returns extracted text, average OCR confidence, line count, per-line bounding boxes, a visual overlay image with detected boxes, table-like row groupings derived from bounding box positions, and common key-value pairs.

2. LLM Integration

The backend uses an open-source Hugging Face causal language model configured through `MODEL_NAME`. The default configuration uses `Qwen/Qwen2.5-1.5B-Instruct`. No proprietary AI APIs are used.

The LLM is loaded lazily and reused for correction, classification, structured extraction, and document question answering. Prompts are constrained to return plain corrected text or valid JSON depending on the route.

3. Information Extraction Approach

The extraction service asks the LLM to return a fixed JSON schema containing common document fields such as document title, name, date, invoice number, total amount, address, phone, email, PAN, GST, Aadhaar, and a `dynamic_fields` object.

`dynamic_fields` captures document-specific values such as invoice line items, vendor names, resume skills, identity-document dates, and bank-statement balances. The backend defensively extracts the first JSON object from the LLM response and returns empty values when parsing fails.

4. Classification Approach

The classifier uses the LLM for document type prediction and confidence. A keyword-based heuristic classifier acts as a reliability guard when the LLM returns malformed output, an unsupported document type, or a weak confidence. Supported classes include Invoice, Receipt, Resume, Aadhaar Card, PAN Card, Driving License, Passport, Bank Statement, and Other.

5. Validation Logic

Validation is format-based and intentionally avoids external paid services or identity verification APIs. The service checks provided values for email, Indian phone number, date, PAN, Aadhaar, GST, total amount, and selected dynamic fields such as subtotal, tax amount, due date, balances, and account number. Missing optional fields are skipped instead of being marked invalid, which keeps receipts, resumes, invoices, and identity documents from failing validation for fields that are not relevant to that document type.

The validation output is a field-by-field boolean map that can be exported as JSON or CSV from the frontend.

6. Sample Documents

The `samples/` folder contains synthetic review documents for multiple classes: invoice, receipt, resume, Aadhaar-style identity card, and bank statement. These documents are safe for demos because they do not contain private user data.

7. Challenges And Limitations

OCR quality depends heavily on scan clarity, skew, noise, resolution, and text language. Local LLM inference can be slow on CPU-only systems, and first-time model startup may require a large Hugging Face download.

Recommended local hardware is at least 16 GB RAM, with 32 GB preferred for a smoother Qwen demo. GPU acceleration significantly improves model-backed correction, extraction, classification, and Q&A latency.

The project includes deterministic tests for routing, parsing, validation, and service behavior. Real end-to-end OCR/LLM quality should still be verified on representative sample documents before final submission.